

Lab 3 Answer Key (Instructor Only)

Module	ET4047
Date	27/09/2026
Lab No.	Lab 3 - Answer Key
Author	Luke Griffin

Model answers for every **Your Turn** exercise in the Lab 3 handout. Other correct solutions are possible. Any program that gives the right result, fits in the 18 memory cells and halts correctly should get full marks. Addresses assume the file is loaded as written: lines go in from address 0, and the model solutions do not use NOP padding.

Task 1

1.1 Each instruction is 16 bits: an 8-bit opcode plus an 8-bit operand. Each memory address holds 8 bits, so an instruction uses **two** addresses, and the PC adds 2 after every fetch. HLT is the 4th instruction, at address 6.

1.2

Line 3 becomes	Addressing mode	ACC
LOD A	direct (label)	5 , the contents of address 34
LOD 34	direct (numeric address)	5 , the same cell as A
LOD #34	immediate	34 , the number itself
LOD #A	immediate (label)	34 , the address of A used as a number

1.3

Address	Instruction	Opcode byte	Operand byte
0	LOD #5	10000111	00000101 (the number 5)
4	LOD A	00000111	00100010 (address 34)

Only the leftmost bit (the **immediate flag**) differs: 1 = immediate, 0 = direct. The opcode 0000111 (LOD) is the same.

1.4 Expected: A = 5, B = 9.

	LOD #5	; #5 -> ACC
	STO A	; ACC -> [A]
	LOD #9	; #9 -> ACC
	STO B	; ACC -> [B]
	HLT	
A:	0	
B:	0	

1.5 A third variable is needed because the ACC holds only one value. Expected: A = 8, B = 3.

```

    LOD A      ; [A] -> ACC
    STO TEMP   ; ACC -> [TEMP]    keep a copy of A
    LOD B      ; [B] -> ACC
    STO A      ; ACC -> [A]      A = old B
    LOD TEMP   ; [TEMP] -> ACC
    STO B      ; ACC -> [B]      B = old A
    HLT
A:      3
B:      8
TEMP:   0

```

Task 2

2.1 SUM = -1 (3 - 4) and ACC = -1. **N = 1** because the result is negative, and **Z = 0** because it is not zero. Watch for students who expect +1: the order is ACC - [B].

2.2

Instruction	ACC after	Instruction	ACC after
LOD #4	4	MUL #3	6
MUL #5	20	SUB #1	5
ADD #3	23	STO Y	5 (Y = 5)
STO X	23 (X = 23)	MUL X	115
LOD #2	2	MUL #2	230, then STO W gives W = 230

2.3 By hand: $(3 + 20) \times (2 - 1) \times 4 = 23 \times 1 \times 4 = 92$. Expected X = 23, Y = 1, W = 92. This is the lecturer's Example_2.cpuvs.

```

; W = (3+4*5)*(8/4-1)*4
    LOD #4      ; Find X = 3+4*5
    MUL #5
    ADD #3
    STO X
    LOD #8      ; Find Y = 8/4-1
    DIV #4
    SUB #1
    STO Y
    MUL X       ; Find X*Y
    MUL #4      ; Find X*Y*4
    STO W       ; Save result to W
    HLT
X:      0
Y:      0
W:      0

```

2.4 Expected AVG = **7**. With C = 11 the sum is 22, and $22 / 3$ gives **7**, not 7.33. DIV works only with integers, so the remainder is discarded. Please confirm this once in the simulator.

```
        LOD A      ; [A] -> ACC
        ADD B      ; ACC + [B] -> ACC
        ADD C      ; ACC + [C] -> ACC    (21)
        DIV #3     ; ACC / #3 -> ACC
        STO AVG    ; ACC -> [AVG]
        HLT
A:      4
B:      7
C:     10
AVG:    0
```

2.5 The immediate operand is only 8 bits (I8, -128 to 127), so 300 does not fit.
Method 1: store 300 as data (16-bit) and use direct addressing:

```
        LOD BIG    ; [BIG] -> ACC    (direct: 300 is stored as 16-bit data)
        HLT
BIG:    300
```

Method 2: build 300 from small immediate values:

```
        LOD #100   ; #100 -> ACC
        MUL #3     ; ACC * #3 -> ACC
        HLT
```

Task 3

3.1 FLAG = **0**. JNZ at address 4 is taken and jumps to address 12 (NOTZERO), so addresses **6, 8 and 10** are skipped (LOD #1, STO FLAG and JMP END). Afterwards Z = 0.

3.2 The zero case becomes the jump target:

```
        LOD X      ; [X] -> ACC
        CMP #0     ; ACC == 0?
        JZ ISZERO  ; If yes, jump to ISZERO
        LOD #0     ; ELSE: 0 -> [FLAG]
        STO FLAG
        JMP END
ISZERO: LOD #1     ; THEN: 1 -> [FLAG]
        STO FLAG
END:    HLT
X:      0
FLAG:   0
```

3.3 Results: $X = 7$ gives $SMALL = 1$, $X = 10$ gives $SMALL = 0$ ($10 - 10 = 0$ is not negative), and $X = 12$ gives $SMALL = 0$. Check $X = 10$ in student work, as it is the edge case.

```

        LOD X          ; [X] -> ACC
        CMP #10        ; ACC - 10: negative when X < 10 (sets N)
        JN LESS        ; If X < 10, jump to LESS
        LOD #0          ; ELSE: 0 -> [SMALL]
        STO SMALL
        JMP END
LESS:   LOD #1          ; THEN: 1 -> [SMALL]
        STO SMALL
END:    HLT
X:      7
SMALL:  0

```

Task 4

4.1 $RESULT = 2$ and $Z = 0$, because CMP gives $4 - 3 = 1$, which is not zero, so JNZ jumps to $NOTEQ$.

4.2 Without $JMP\ END$, the CPU "falls through" from the $THEN$ block into the $ELSE$ block. It stores 1 and then immediately overwrites it with 2, so $RESULT$ is always **2**, even when $A = B$. This is why $IF-THEN-ELSE$ needs an unconditional jump.

4.3 Pseudo code: $IF\ (X == 3)\ THEN\ Y = Y + 5\ ELSE\ Z = Z + 2\ ENDIF; X = 8.$

Starting X	Block that runs	X after	Y after	Z after
3	THEN	8	5	0
5	ELSE	8	0	2

X is always 8, because $X = 8$ comes after $ENDIF$. The PC is 24 at the end, matching the lecture's "after execution" screenshot.

4.4 Expected $MAX = 9$ in both tests. After CMP the ACC still holds A , so $STO\ MAX$ stores A without reloading it. Accept solutions that reload A . If $A = B$, either value is correct.

```

        LOD A          ; [A] -> ACC
        CMP B          ; ACC - [B] (ACC still holds A afterwards)
        JN BBIG        ; If A < B, jump to BBIG
        STO MAX        ; THEN: ACC -> [MAX] (A is still in ACC)
        JMP END
BBIG:   LOD B          ; ELSE: [B] -> ACC
        STO MAX        ;       ACC -> [MAX]
END:    HLT
A:      4
B:      9
MAX:    0

```

Task 5

5.1 SUB #1 runs **6** times. When JNZ jumps, the PC is loaded with **2**, the address of LOOP.

5.2 With $N = 0$, SUB #1 makes $N = -1$ before the first test, so N is never 0 when CMP runs. The program counts down through negative numbers and does not halt normally. This is a REPEAT-UNTIL loop, which always runs the body at least once. The fix is the WHILE-DO pattern, which tests first:

```

LOOP:  LOD N      ; [N] -> ACC
        CMP #0    ; WHILE N != 0 (test BEFORE decrementing)
        JZ DONE   ; If N == 0, jump to DONE
        SUB #1    ; ACC - 1 -> ACC
        STO N     ; ACC -> [N]
        JMP LOOP  ; Jump back to LOOP

DONE:   HLT
N:      0

```

5.3

Pass	COUNT	SUM
1	1	1
2	2	3
3	3	6
SUM for MAX = 5	5	15 (1 + 2 + 3 + 4 + 5)

5.4 WHILE-DO tests **at the top** and must **leave** the loop when the condition becomes false ($COUNT == MAX$), so it jumps forwards with JZ ENDWHILE. The countdown tests **at the bottom** and must **repeat** while the counter is not yet zero, so it jumps backwards with JNZ LOOP.

Task 6

6.1

A	B	W	A	B	W
0	0	0	-1	0	-1
0	-1	-1	-1	-1	-1

6.2 An **OR** gate. By De Morgan's law, $\text{NOT}(\text{NOT } A \text{ AND } \text{NOT } B) = A \text{ OR } B$. Because the operations are bitwise, $A = 5$ (0101) and $B = 3$ (0011) give $W = 0111 = 7$.

6.3 Expected $W = 0$ for $A = B = -1$. Every other combination of 0 and -1 gives -1 .

```
        LOD A      ; [A] -> ACC
        AND B      ; ACC AND [B] -> ACC
        STO T      ; ACC -> [T]
        NOT T      ; NOT([T]) -> ACC
        STO W      ; ACC -> [W]
        HLT
A:      -1
B:      -1
T:      0
W:      0
```

Challenges

C.1 This is the lecturer's Quotient_and_Reminder.cpuvs. Expected $Q = 3$, $R = 2$.

```
        LOD #0     ; Initialisation: 0 -> ACC
        STO Q      ; 0 -> [Q]
        LOD A      ; [A] -> ACC
        STO R      ; [A] -> [R]
LOOP:   LOD R      ; [R] -> ACC
        CMP B      ; Test R < B?
        JN DONE    ; If yes, go to DONE
        SUB B      ; Otherwise ACC - [B] -> ACC
        STO R      ; ACC -> [R]
        LOD Q      ; [Q] -> ACC
        ADD #1     ; ACC + 1 -> ACC
        STO Q      ; ACC -> [Q]
        JMP LOOP   ; Repeat while R >= B
DONE:   HLT        ; Stop execution
A:      17
B:      5
Q:      0          ; 3 after execution
R:      0          ; 2 after execution
```

C.2 Expected RES = **120**. If N starts at 1, the loop still runs once: RES = 1 and N becomes 0. Then CMP #1 is never zero again, so the loop never ends. Fix: test N before multiplying (WHILE-DO), or compare with 1 at the start of the loop.

```
LOOP:  LOD RES      ; [RES] -> ACC
        MUL N       ; ACC * [N] -> ACC
        STO RES     ; ACC -> [RES]
        LOD N       ; [N] -> ACC
        SUB #1      ; ACC - 1 -> ACC
        STO N       ; ACC -> [N]
        CMP #1      ; ACC == 1?
        JNZ LOOP    ; If not, repeat
        HLT
N:      5
RES:    1
```

Review Questions

R1 LOD 34 is direct: it loads the **contents** of address 34, which is the same as LOD A. LOD #34 is immediate: it loads the **number** 34.

R2 CMP updates the flags exactly as SUB would, but leaves the ACC unchanged. It sets up the Z and N flags for the jump without destroying the value being tested.

R3 The **PC**. A jump overwrites the PC with the operand address, so the next fetch comes from there. Jumping to an earlier address repeats instructions, which is a loop.

R4 ADD, SUB, MUL, DIV, AND and CMP. LOD and STO do not update the flags, so after a LOD the flags still describe an older result. CMP is needed to set them for the value just loaded.

R5 The operand field of an instruction is only 8 bits, which gives the range -128 to 127 in two's complement. A data value fills a whole 16-bit memory word (116), which gives -32768 to 32767.